

Virtual Memory

Sistemas de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

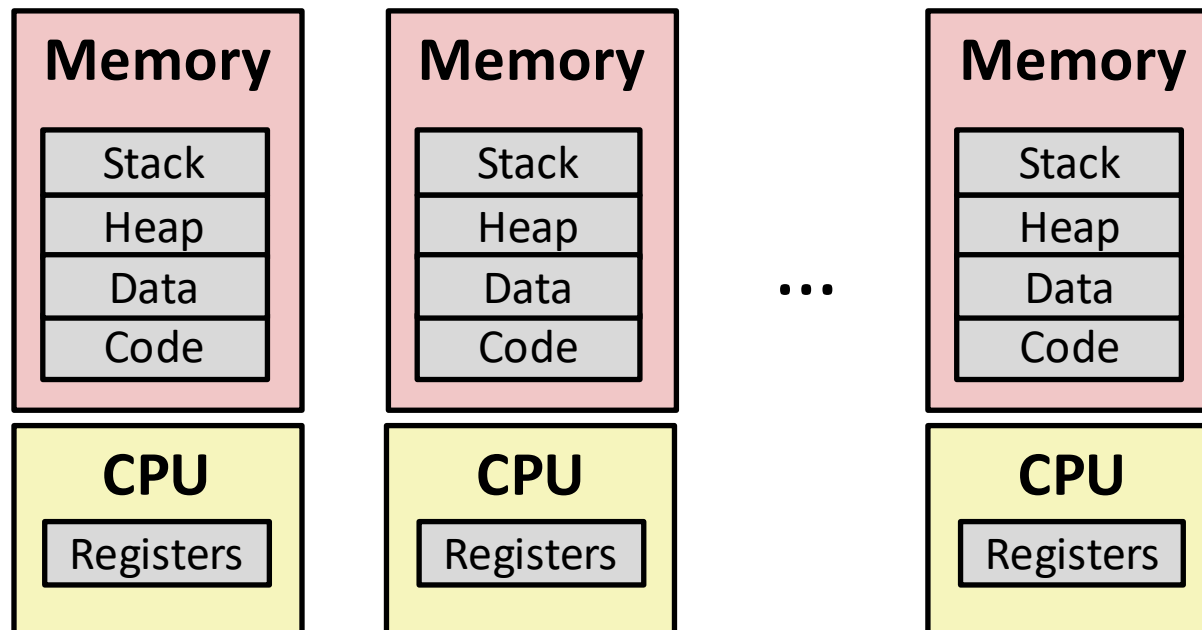
Luís Nogueira (lmn@isep.ipp.pt)

Overview

- Virtual memory (VM)
- VM as a tool for caching
- VM as a tool for memory management
- VM as a tool for memory protection

Multitasking

- We want to have multiple concurrently executing processes
- How can many programs share a computer efficiently?



Virtual memory

- **Virtual memory (VM) is the mechanism that decouples programs from physical memory**
 - Each process executes as if it owns a large, contiguous memory space starting at address zero, regardless of how memory is laid out in RAM
- **This abstraction is essential for safe multitasking**
 - Without it, concurrent programs would interfere with each other through accidental or malicious memory accesses
- **VM is therefore a foundational mechanism for **isolation**, **protection**, and **controlled sharing** in modern OSes**

Virtual memory

- Without hardware support, VM would be far too slow to implement in software alone
- VM is managed **cooperatively** by hardware and the OS
 - Hardware provides fast translation, caching, and protection checks
 - The OS manages page tables, permissions, allocation, and page faults
- Together, they deliver a flexible and efficient abstraction of a large, continuous address space

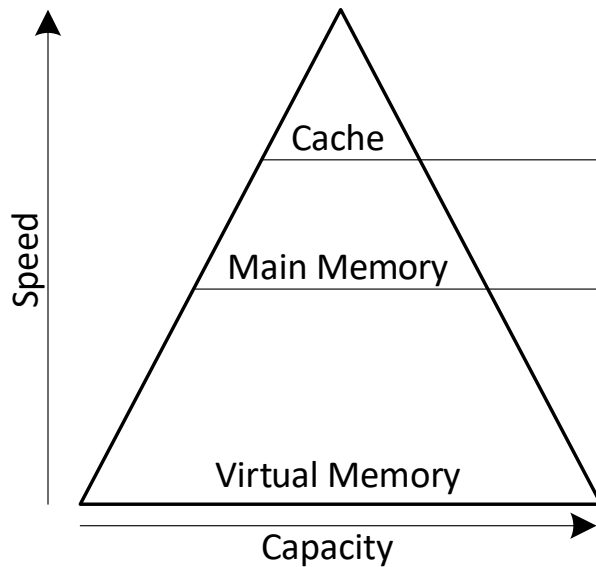
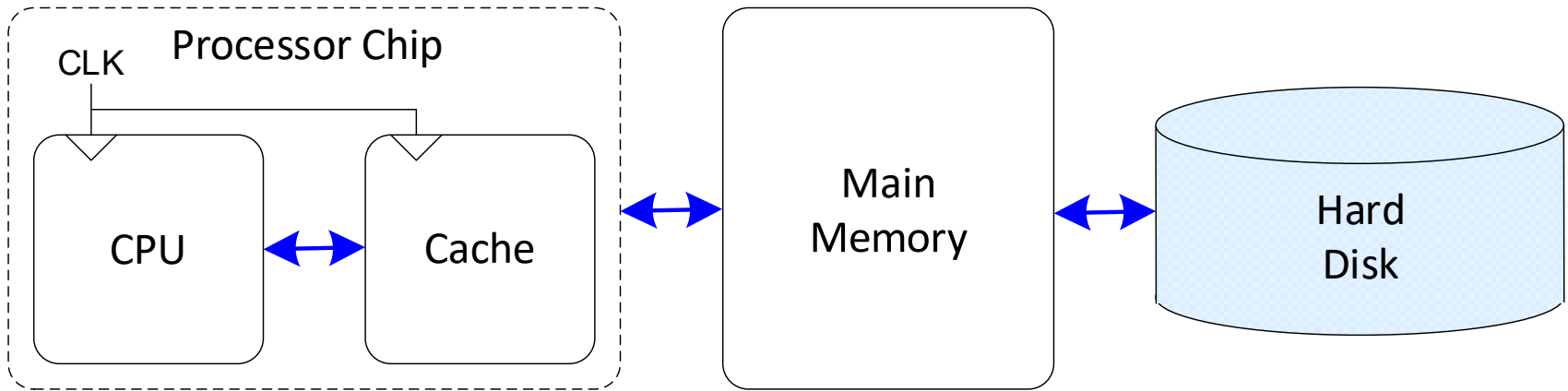
Overview

- Virtual memory (VM)
- **VM as a tool for caching**
- VM as a tool for memory management
- VM as a tool for memory protection

Virtual memory

- Modern systems extend the memory hierarchy beyond main memory by **adding a large, slower level stored on disk**
- Even if a program fits in memory, running several program may not
 - VM allows the virtual address space to be larger than physical memory, with active pages cached in RAM
- Provides the illusion of a huge, private, contiguous address space for every process
 - Typically, up to 4 GB in 32-bit architectures and up to 128-256 TB (and even more in modern designs) in 64-bit systems

Memory hierarchy



Technology	Price / GB	Access Time (ns)	Bandwidth (GB/s)
SRAM	\$100	0.2 - 3	100+
DRAM	\$3	10 - 50	30
SSD	\$0.10	20,000	0.05 - 3
HDD	\$0.03	5,000,000	0.001 - 0.1

The memory hierarchy challenge

■ Speed disparity

- CPU registers are fast (sub-nanosecond), but disks are millions of times slower

■ Cost vs. performance

- SRAM is expensive (\$100/GB) while HDD is cheap (\$0.03/GB)

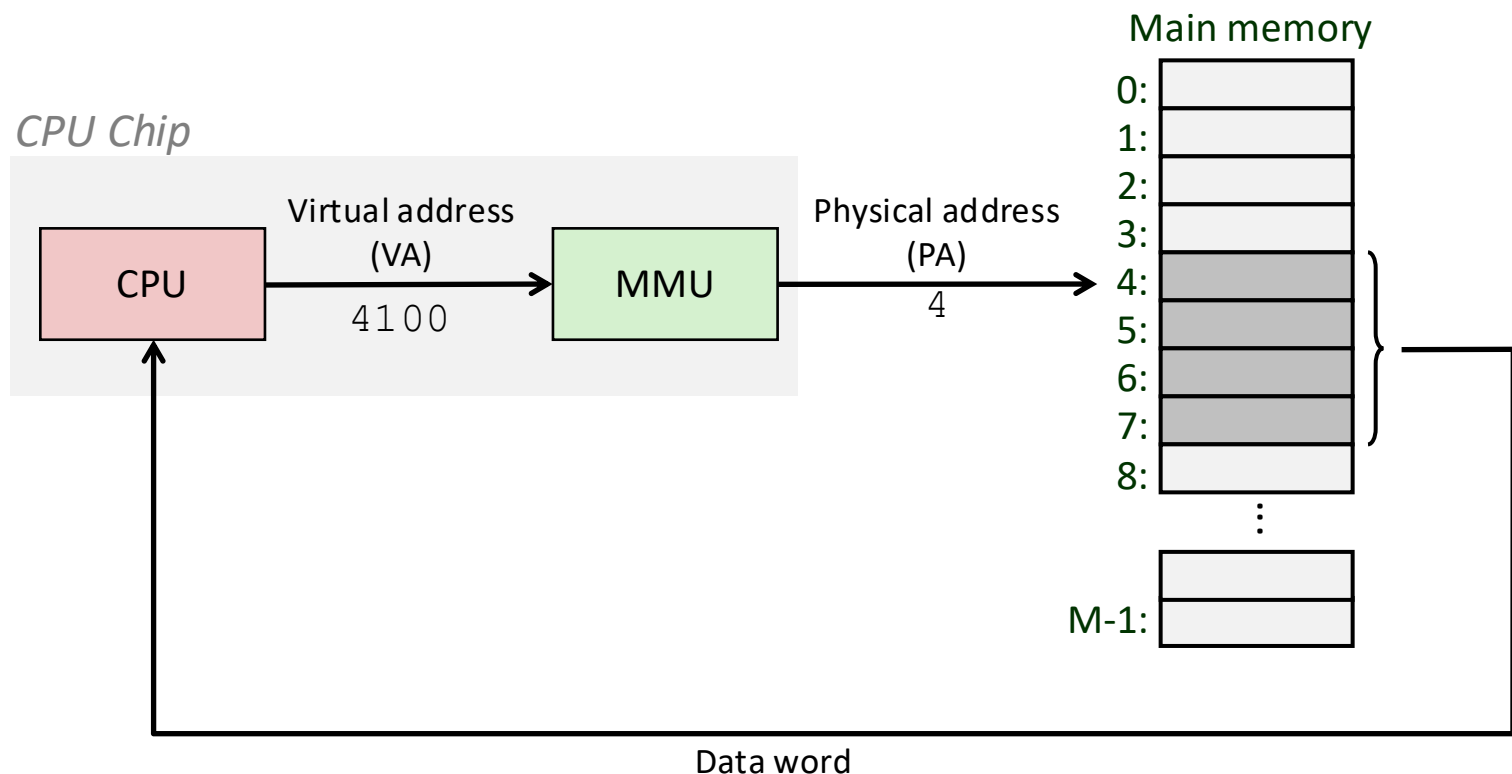
■ VM as a cache system

- The OS must decide **which pages to keep in memory** based on access patterns and locality
- Poor locality or excessive concurrency can lead to thrashing, where useful work stalls due to constant paging

Virtual vs physical addressing

- **Programs generate virtual addresses**
 - Which have no direct meaning to the hardware memory system
- **Physical addresses correspond to actual locations in RAM**
- **Virtual memory provides a **translation layer** between the two**
 - Allowing the OS to change physical memory layout without affecting programs
 - This indirection enables memory relocation, fragmentation tolerance, and independent address spaces for concurrent processes

A system using virtual addressing



- **Physical addresses are completely invisible to programs**
 - The run-time mapping from virtual to physical addresses is done by a hardware device called the memory-management unit (**MMU**)

The Memory Management Unit (MMU)

- The MMU is a hardware component that performs **address translation and permission checking** on every memory access
 - Intercepts every memory access; translates virtual addresses (used by the program) to physical addresses (RAM)
 - Consults the active page table and raises a fault if an access is invalid, preventing a process from accessing or corrupting the memory of another process
- The MMU operates transparently to the user's program
 - Programs operate in a contiguous virtual address space, unaware of fragmented physical memory or other running processes

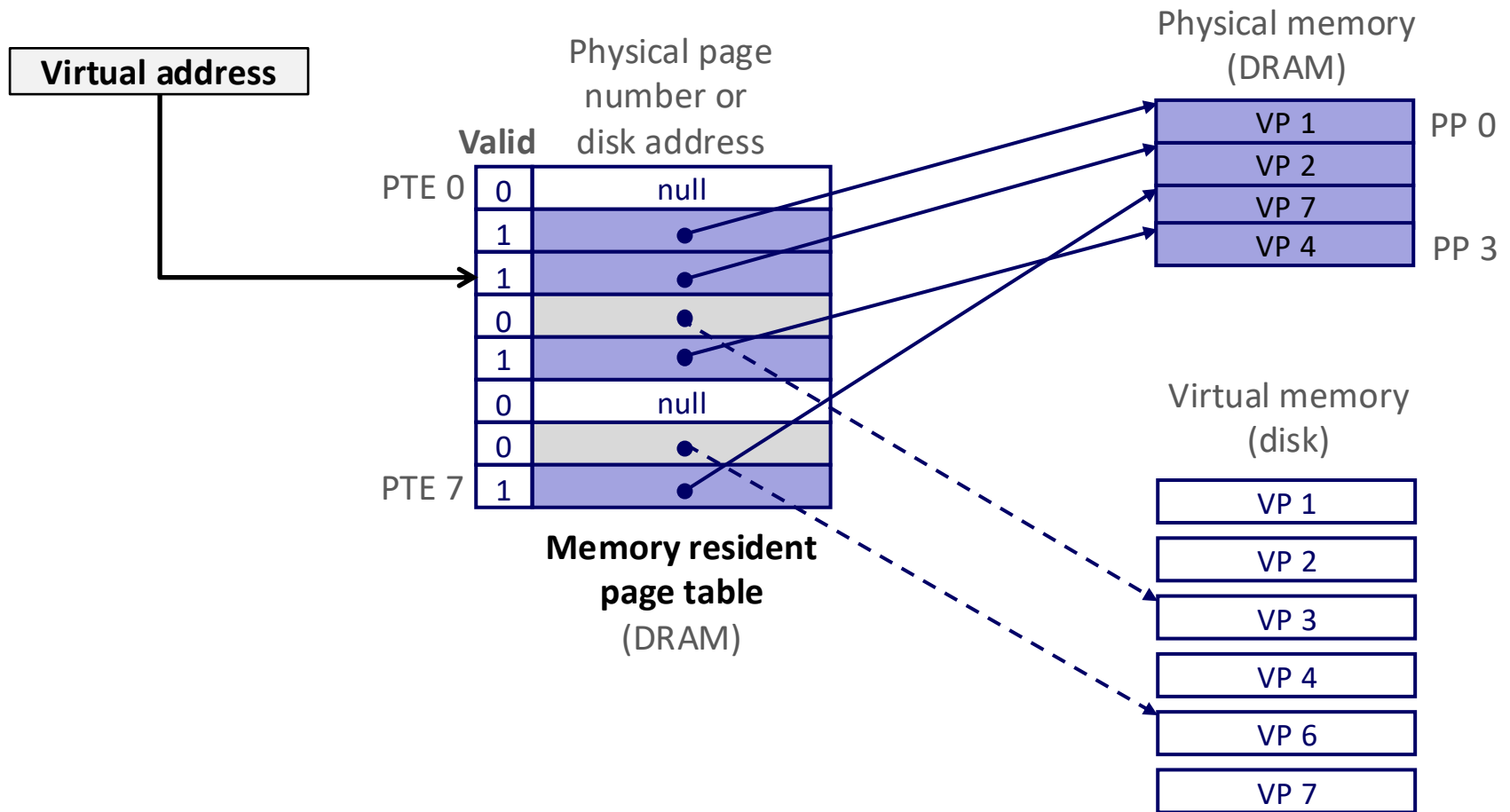
Pages and address translation

- **Virtual and physical memory are divided into fixed-size units called **pages****
 - Typically range from 4 KB to 1 GB, depending on the architecture
- **A virtual address is split into a virtual page number and an offset**
 - The offset selects a byte within a page
 - The page number is translated to a physical page frame
- **Paging simplifies allocation, enables efficient protection, and makes sharing practical at page granularity**

Page tables and Page Table Entries

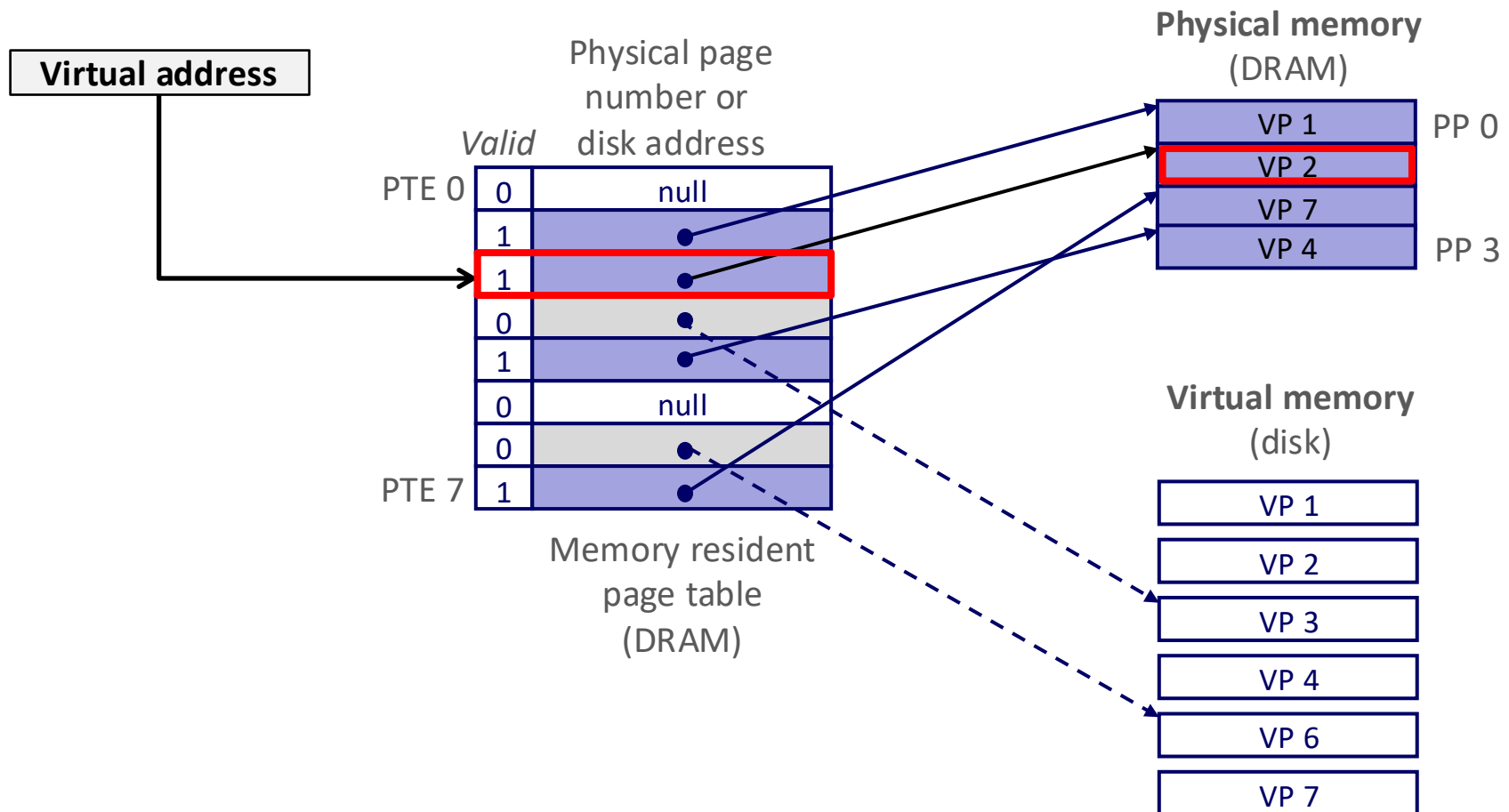
- **Page tables** store the mapping between virtual pages and physical page frames
 - A page table is a **per-process** kernel data structure in DRAM with page table entries (PTEs) that map virtual pages to physical pages
- **Page tables are entirely under OS control**
 - Each entry includes not only the physical page number, but also metadata such as validity and access permissions
 - Every memory access is checked against these permissions by hardware, ensuring that illegal accesses are detected immediately

Page table



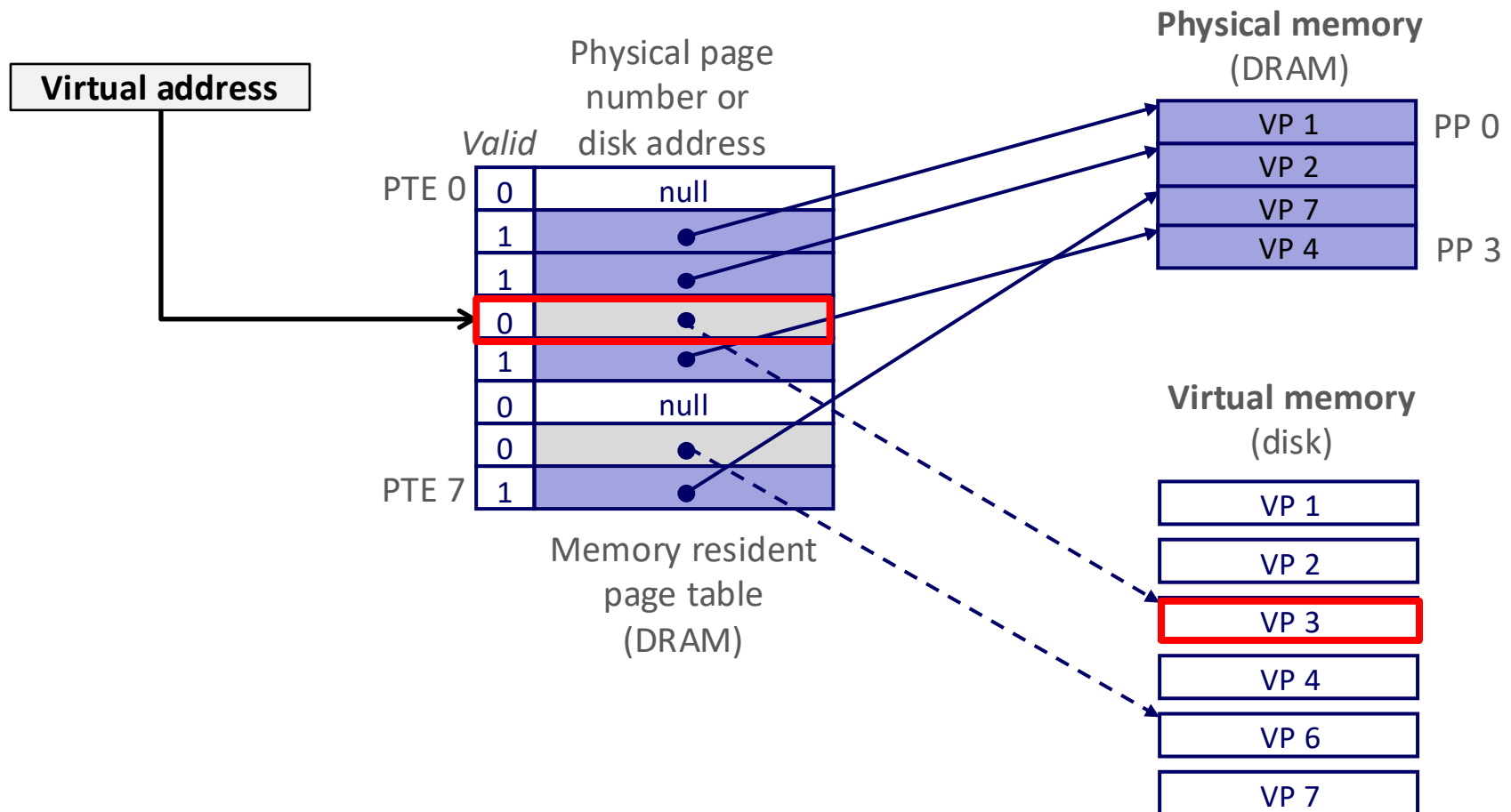
Page hit

- A reference to a VM word that is in physical memory



Page fault

- A reference to a VM word that is unmapped, not present in memory, or accessed with invalid permissions



Page faults and OS intervention

- **When the MMU fails to translate a virtual address, the hardware “traps” (synchronous interrupt) into the OS**
 - The OS determines the fault type: major fault (valid access, but page is on disk/swap) vs. fatal fault (illegal access/protection violation)
 - (more details in the next lecture)
- **Page faults are precise and synchronous**
 - Page faults occur at the exact instruction that triggered the illegal or missing access
 - This precision is vital for debugging memory safety bugs (e.g., Use-After-Free, null pointer dereference)

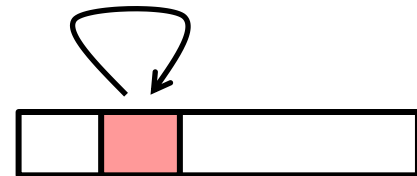
Locality to the rescue

■ Virtual memory seems terribly inefficient, but it works because of **locality**

- At any point in time, programs tend to access a set of active virtual pages called the **working set**
- Programs with better temporal locality will have smaller working sets

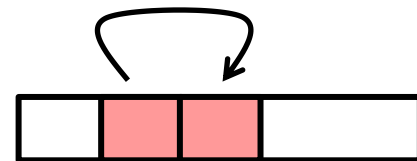
■ **Temporal** locality

- Recently referenced items are likely to be referenced again soon



■ **Spatial** locality

- Items with nearby addresses tend to be referenced close together in time



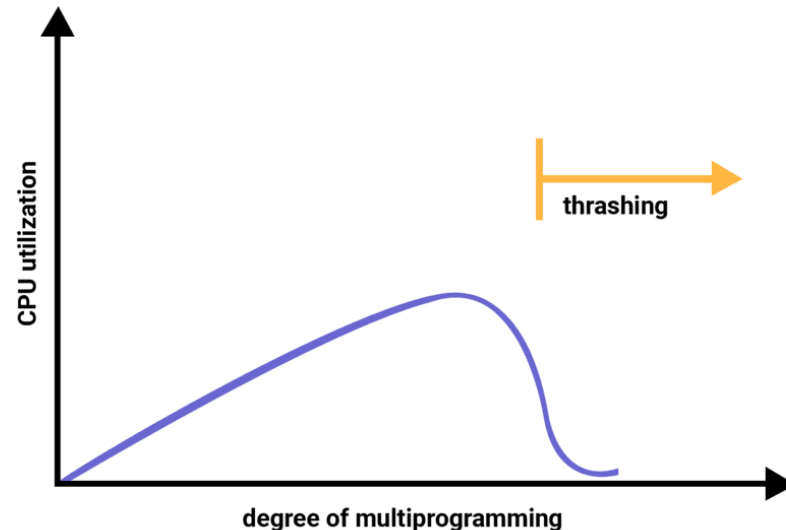
Locality to the rescue

■ If working set size < main memory size

- Good performance for a single process once initial cold misses are handled

■ If working set size > main memory size

- **Thrashing**: Severe performance degradation due to continuous page swapping



Translation looks slow...

- **The MMU accesses memory twice:**

- Once to get the PTE for translation
- Again, for the actual memory request

- **What can we do to make this faster?**

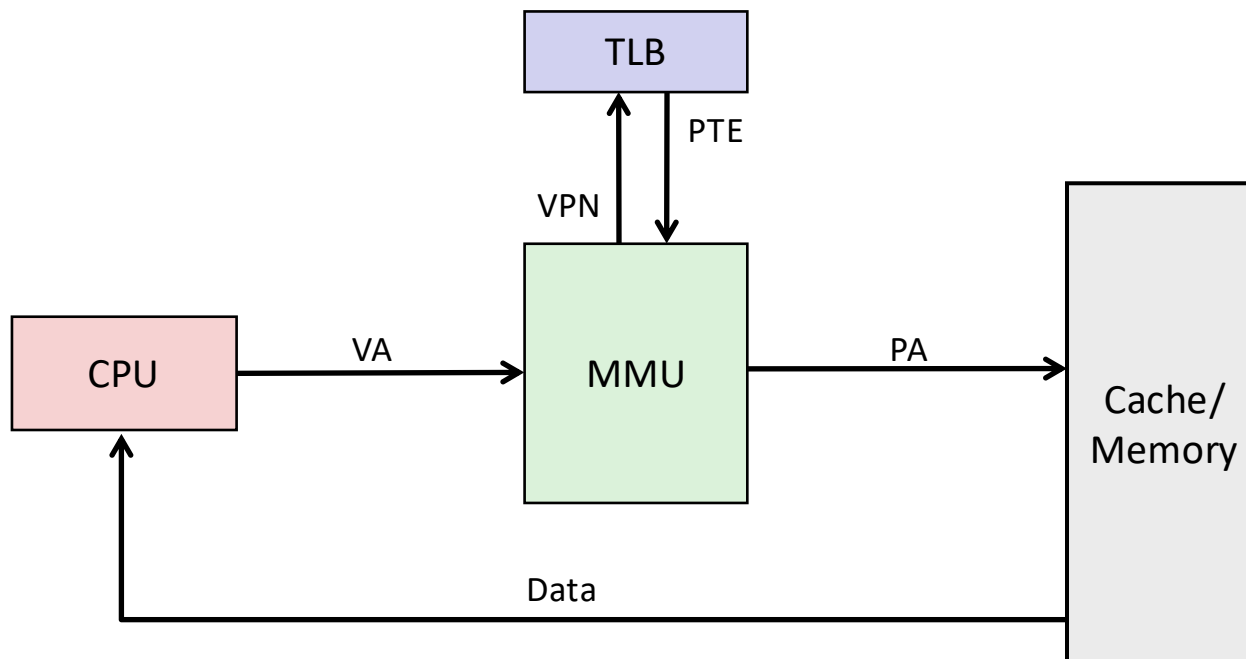
- **Add another cache!**

- **A TLB hit allows address translation in a single cycle, while a miss requires a page table walk**

- In concurrent systems, frequent context switches and large working sets increase TLB pressure, making TLB behavior a key performance factor

Translation Lookaside Buffer

- A small cache dedicated to storing mappings from virtual addresses to physical addresses (PTEs)
 - MMU consults the TLB for each address as its first action
 - If there is a **TLB hit**, it does not need to fetch anything from the page table



VA: virtual address, PA: physical address, PTE: page table entry, PTEA = PTE address

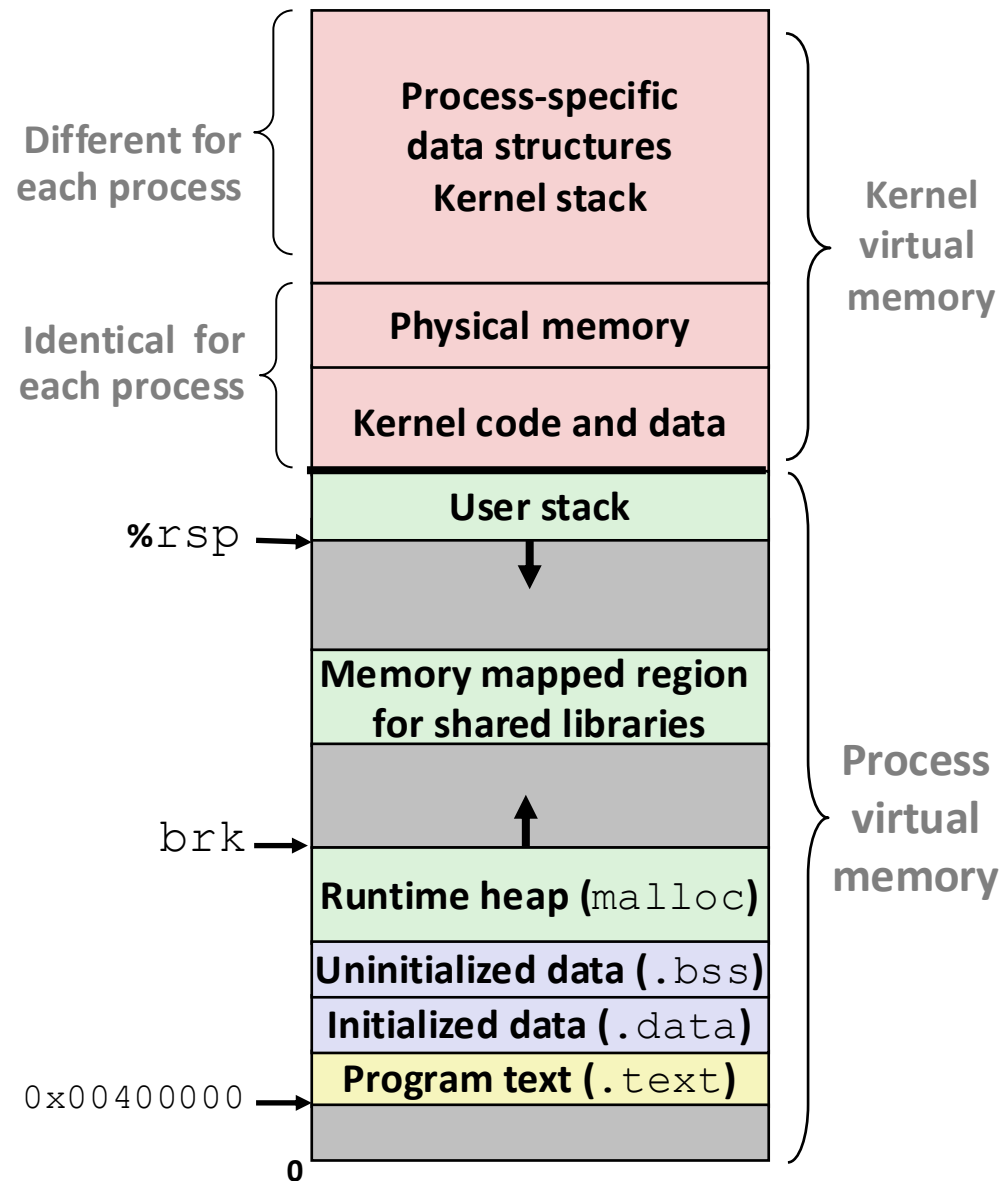
Overview

- Virtual memory (VM)
- VM as a tool for caching
- **VM as a tool for memory management**
- VM as a tool for memory protection

Address spaces and process isolation

- **Each process runs in its own virtual address space**
 - Defined by a page table managed by the OS
- **Two processes may use the same virtual address while referring to entirely different physical memory**
 - This ensures that concurrent processes are isolated by construction
 - Memory races between processes are impossible by default, unless the OS explicitly allows sharing
- **VM therefore acts as the first and strongest concurrency control mechanism in the system**

Private linear address space



Isolation by construction

■ Memory isolation (hardware-enforced)

- Enforced by hardware (MMU)
- Configured by the OS

■ Isolation does not rely on:

- Locks or synchronization
- Programmer discipline
- Software runtime checks

■ Why this works

- Every memory access is implicitly validated by the MMU
- Illegal accesses trigger a hardware exception
- Faulting instructions never complete

Sharing memory

■ Systems still need to share memory

- Communication between processes
- Sharing code and data
- High-performance I/O

■ Key idea

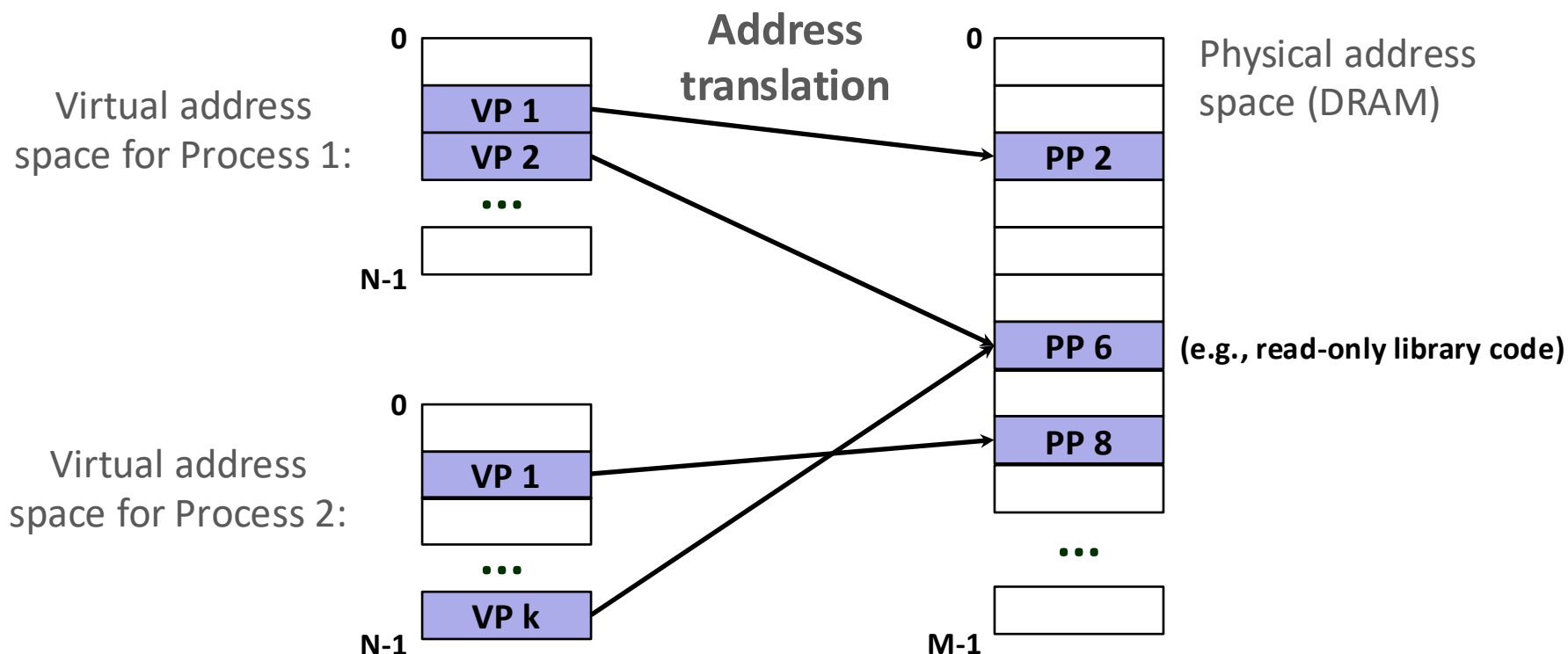
- Sharing is explicit and controlled
- Still enforced by hardware and OS

■ Isolation first — sharing by design

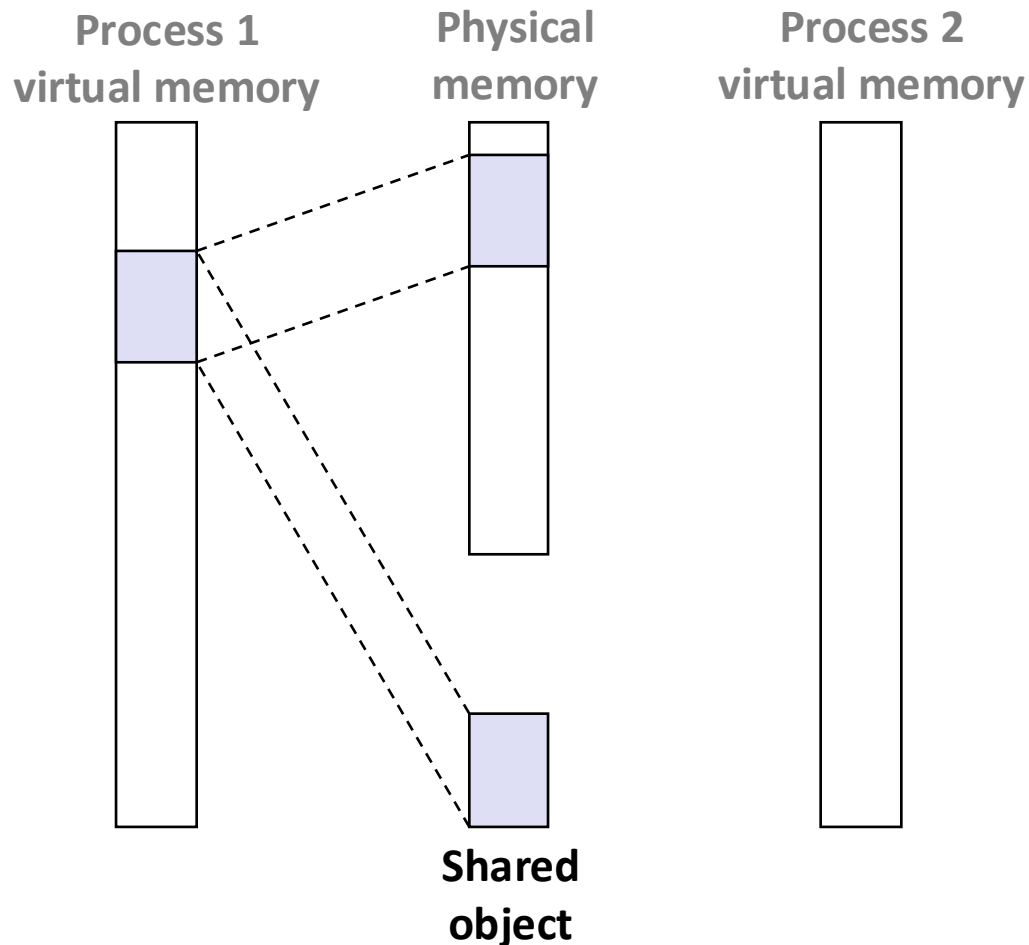
- VM defaults to isolation
- Sharing is an exception, not the rule

Sharing memory

- **Paged VM enables sharing at the granularity of a page**
 - By allowing page tables to **point to the same physical addresses**



Sharing memory



- **Process 1 maps the shared object (on disk)**
 - VM areas initialized by associating them with disk objects
 - Called **memory mapping**

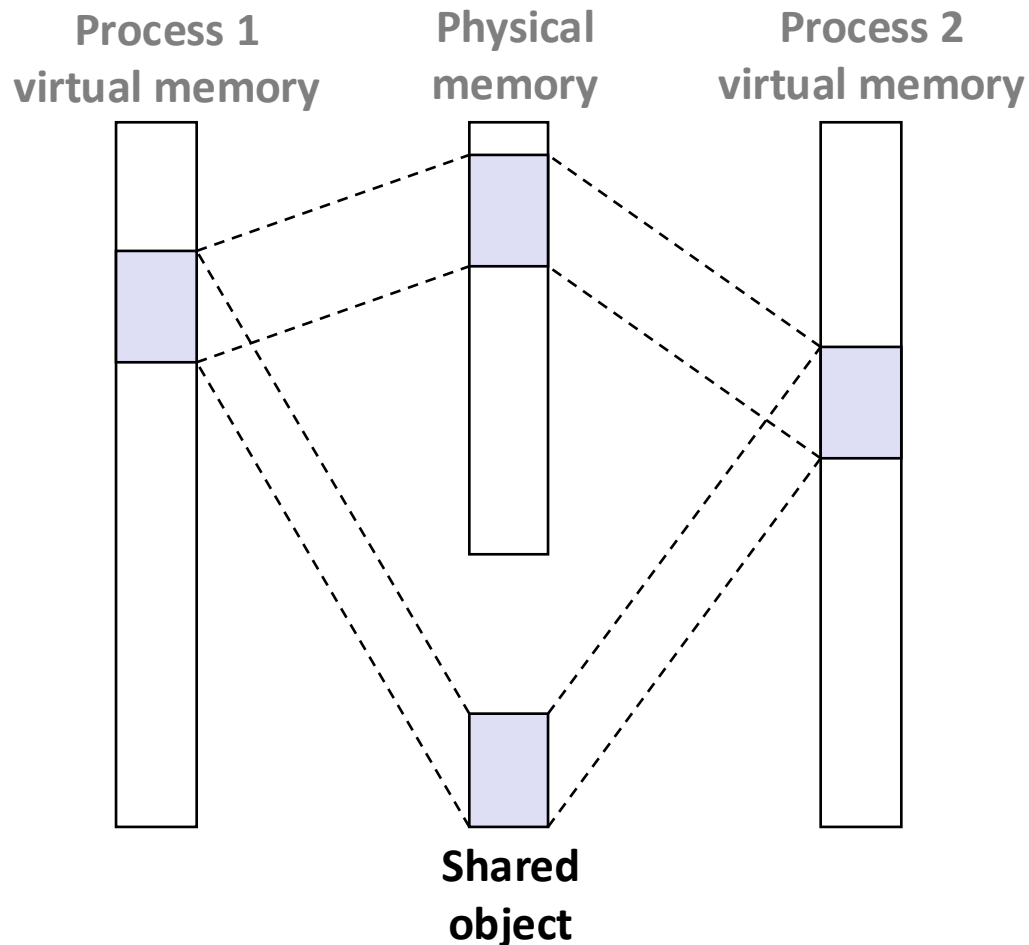
User-level memory mapping

- Sharing is safe because permissions are enforced per mapping, not per physical page

```
void *mmap(void *start, int len, int prot, int flags,  
           int fd, int offset);
```

- Map `len` bytes starting at offset `offset` of the file specified by file description `fd`, preferably at address `start`
 - **start**: may be 0 for “pick an address”
 - **prot**: `PROT_READ`, `PROT_WRITE`, `PROT_EXEC`, ...
 - **flags**: `MAP_ANON`, `MAP_PRIVATE`, `MAP_SHARED`, ...
- Return a pointer to start of mapped area
 - (may not be `start`)

Sharing memory

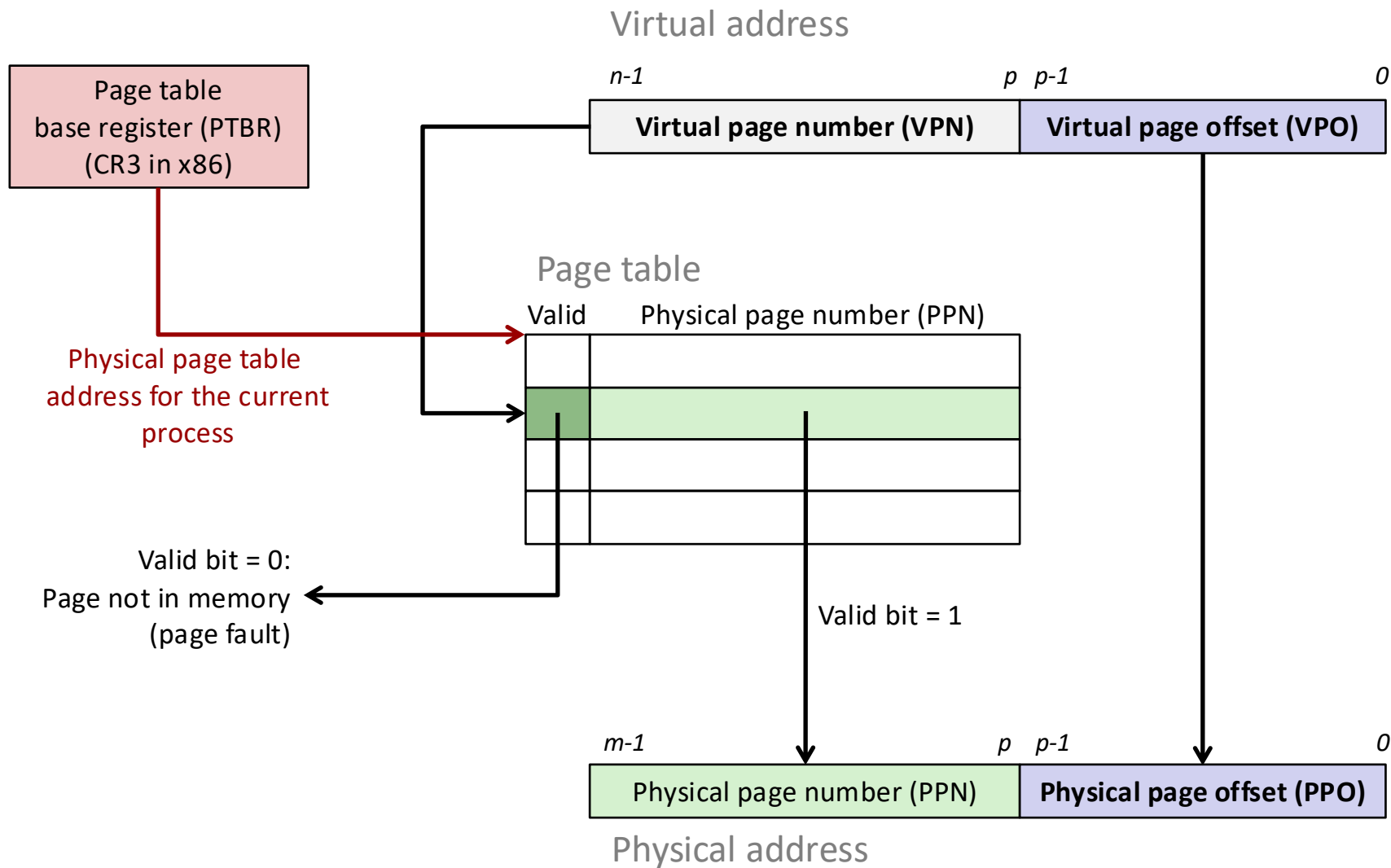


- **Process 1 maps the shared object (on disk)**
- **Process 2 maps the same shared object**
- **Notice how the virtual addresses can be different**

Context switches and VM

- **When the OS switches between processes, it not only saves and restores registers but also changes the active page table**
 - Recall that a page table is a **per-process** kernel data structure in DRAM
 - This instantly changes the visible memory without copying any data
- **VM makes context switches fast and safe**
 - Each process resumes execution with its own memory view, even though the same physical RAM is being reused
- **A page-table base register (PTBR) points to the page table**
 - Changing page tables primarily requires updating the PTBR (e.g., CR3 on x86), substantially reducing context-switch time (a topic for the next class)

Context switches and VM



Overview

- Virtual memory (VM)
- VM as a tool for caching
- VM as a tool for memory management
- **VM as a tool for memory protection**

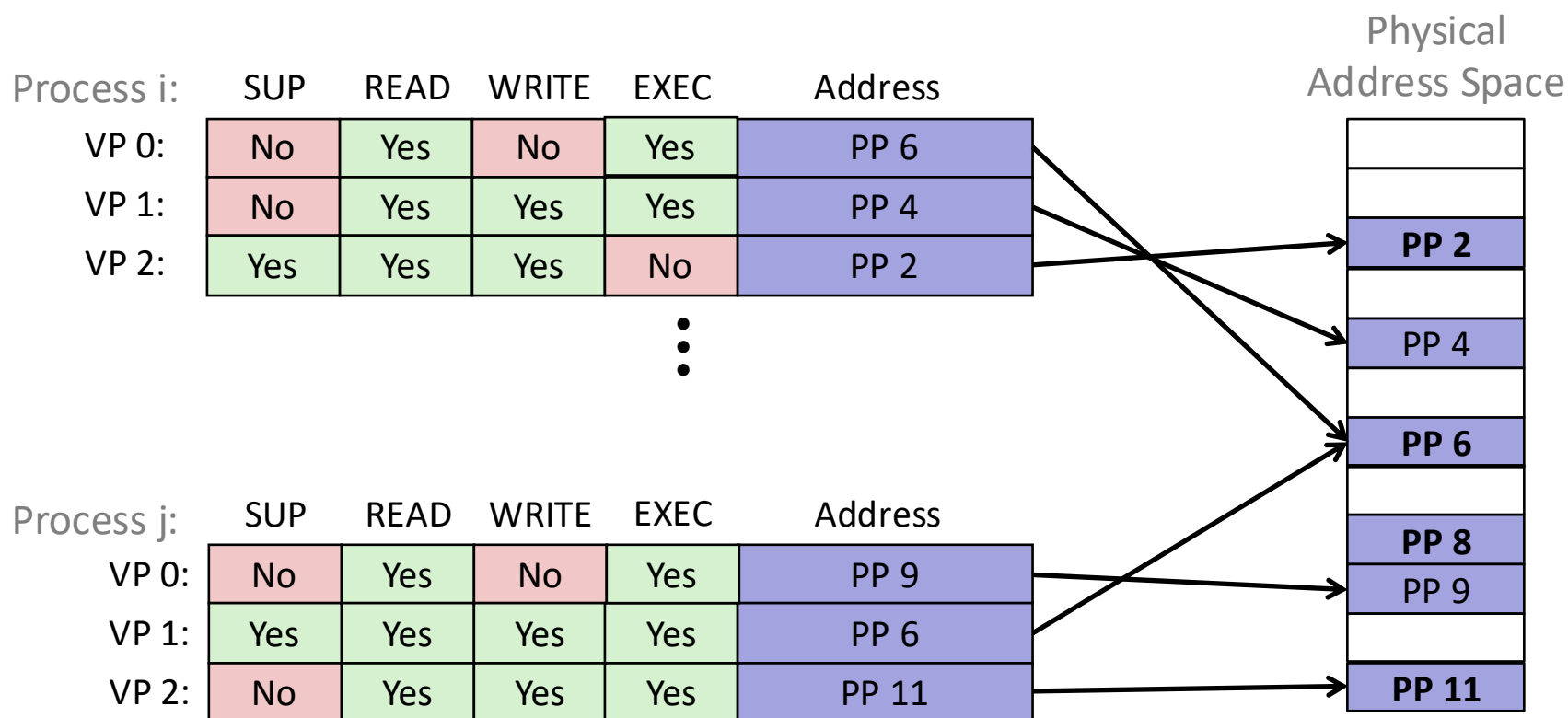
Memory protection and privilege levels

- **Each process generates memory requests without worrying about safety**
 - Each process can only interact with memory space via virtual memory
 - The OS controls which physical memory region is accessible by each process
- **Page permissions enforce critical safety guarantees**
 - Code pages can be read but not written, data pages can be non-executable, and kernel memory is inaccessible to user processes
- **These protections are enforced by the MMU, not by software checks**
 - As a result, even buggy or malicious concurrent programs cannot bypass memory isolation

Memory protection and privilege levels

■ Extend PTEs with **metadata flags** about each page

- MMU checks these bits on each access



What It solves and what it doesn't

- **VM provides isolation, protection, controlled sharing, and predictable failure on bugs**
 - It defines the boundary between safe concurrency (processes) and unsafe concurrency (threads sharing memory)
- **However, VM does not provide synchronization, ordering, or atomicity**
 - Once memory is shared, **correct concurrent behavior still requires locks, atomics, and careful design**

Summary

- **Virtual memory consists of hardware and software resources with three basic functions**
- **Paging**
 - Allows a process to use more memory than physically exists
- **Reallocation**
 - Ensures that each process has its own contiguous address space that starts at zero
- **Protection**
 - Prevents a process from using a memory address that does not belong to it

Summary

■ Programmer's view of virtual memory

- Each process has its own private linear address space
- Cannot be corrupted by other processes

■ System view of virtual memory

- Uses memory efficiently by caching virtual memory pages
- Simplifies memory management and programming
- Simplifies protection by providing a convenient point to check permissions

■ Implemented via combination of hardware and software

- MMU, TLB, exception handling mechanisms part of hardware
- Page fault handlers, TLB management performed in software